

06) – Técnicas de Teste Caixa-Branca (Teste Estrutural)

Prof. Ivan Granja
Disciplina: Verificação e validação de Software

Teste Estrutural (Teste Caixa-Branca)

- Técnica complementar ao teste funcional
- Baseia-se na estrutura interna do software e de sua implementação

- Caracterizar que um determinado conjunto de componente elementares (comandos, ramos condicionais, caminhos) foram exercitados (cobertos) por um conjunto de casos de teste

Teste Estrutural (Teste Caixa-Branca)

- Útil para os seguinte tipos de defeitos:
  - Expressões lógicas e asserções incorretas
  - Tipográficos (trocar > por um >=)
  - Seleção Incorreta de Caminhos
  - Em casos especiais, raros ou que são pouco executados
  - Em situações de mascaramento de defeitos

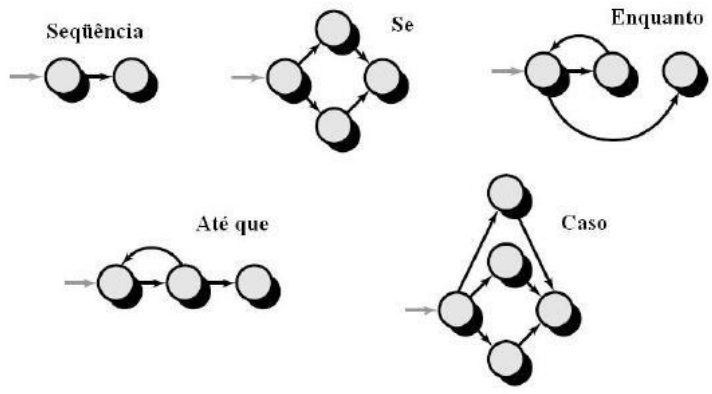
Teste Estrutural (Teste Caixa-Branca)

- Exige instrumentação do programa fonte
  - Fazer com que o programa deixe rastros de sua execução (em um arquivo texto, por exemplo)
  - Técnica Rudimentar: Para cada comando, inserir outro que indica ou não sua execução
  - Técnica Mais Utilizada: Grafo de Programa ou Grafo de Fluxo de Controle (GFC)
    - Exige transformar um Programa em um GFC
- Os casos de teste são então submetidos ao programa modificado (instrumentado) e re compilados
  - Quando submetidos aos casos de teste o programa instrumentado deixa o rastro do caminho executado

Técnicas de Teste Estrutural – O Grafo de Fluxo de Controle (GFC)

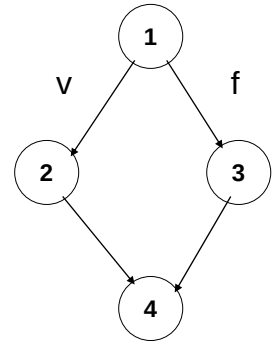
- É um Grafo Dirigido (dígrafo), com um único nó de entrada e um único nó de saída.
- Gerado da decomposição de um programa em blocos de comandos, ligando-os através de arcos.
- Elementos estruturais do GFC:
  - **Nós**: Blocos de Comandos
  - **Arcos**: Transferência de fluxo entre blocos
  - **Caminho**: Sequência de nós $n_1,n_2,n_3,...,n_m$, desde que haja um arco ligando n_i a n_{i+1}.
  - **Regiões:** Áreas do Grafo delimitadas por arcos e nós

Técnicas de Teste Estrutural Notação Básica do GFC



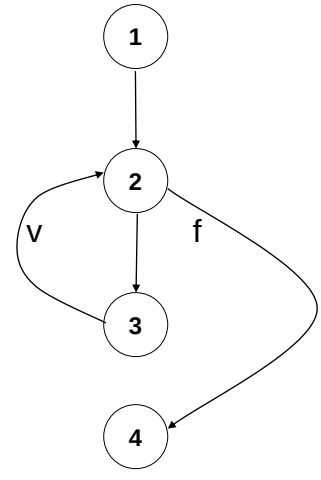
If-Then-Else

comando-anterior {1}
if (condição) {1}
    bloco 1 {2}
else
   bloco 2; {3}
end-if {4}



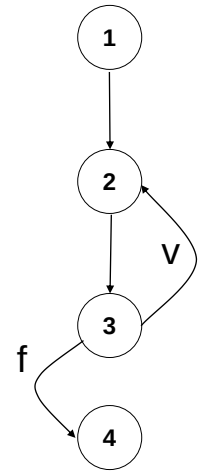
While

comando-anterior {1}
while (condição) {2}
    bloco {3}
end-while {4}

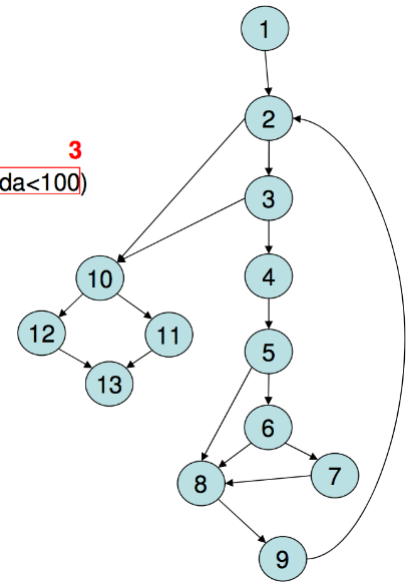


Do.....While

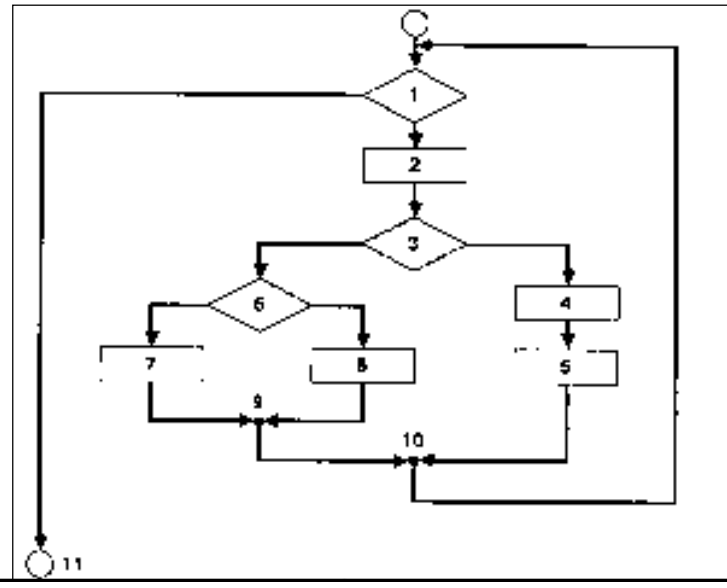
comando-anterior {1}
do
  bloco {2}
while (condição) {3}
próximo-comando {4}



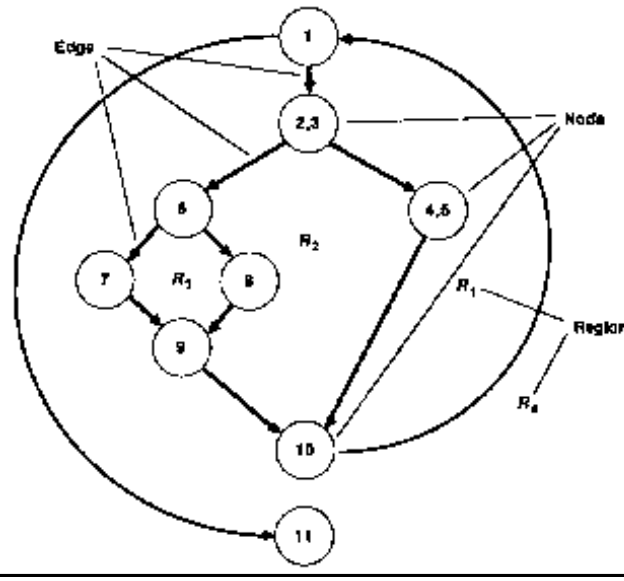

```
Procedimento média(valor[])
i=1;                                          1
soma=0;
total.entrada=0;
total.válidas=0
                                  2                    3
Faça-Enquanto (valor[i]≠-999 E total.entrada<100)
    incremente total.entrada de 1;            4
    Se (valor[i]≥min E valor[i]≤max)
    Então
7           incremente total.válidas de 1;
            soma = soma + valor[i]
    Fim-Se
8   incremente i de 1;
Fim-Enquanto 9
Se total.válidas>0 10
Então média = soma/total.válidas; 11
Senão média = -999; 12
Fim-Se 13
Fim média
```



Técnicas de Teste Estrutural - Fluxograma

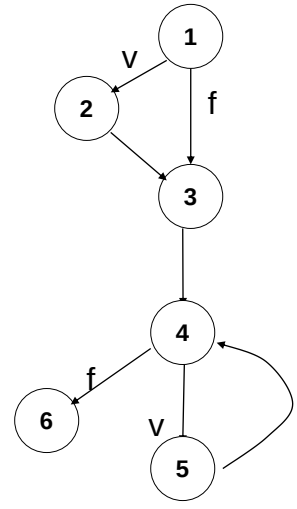


Técnicas de Teste Estrutural – Grafo Equivalente ao Fluxograma



Técnicas de Teste Estrutural Exemplo de GFC

```
Nó Comandos
1 int mdc(int a, int b)
1 { int r,aux;;
1 if (a < b) // inverte a e b
2 { aux = a;
2   a = b;
2   b = aux; }
3 r = a % b;
4 while (r!=0)
5 {   a = b;
5     b = r;
5     r = a % b; }
6  return b;
6 }
```



Técnicas de Teste Estrutural Critérios Estruturais Clássicos

- Consideram apenas as informações do fluxo de controle do programa

Exemplos:

- Todos os Nós
- Todos os Arcos
- Todos os Caminhos (Teste Exaustivo)
- Todos os Caminhos Independentes

Técnicas de Teste Estrutural Critérios Estruturais Clássicos

- Todos-Nós
  - Exige que os casos de teste exercitem todos os nós pelo menos uma vez
  - Se todos os Nós forem Executados, todos os comandos terão sido exercitados pelo menos uma vez:

- Exemplo: Entrada Caminho Completo Executado Saída Esperada
  - Caso de teste [(4,30) , (1,2,3,4,5,4,6) , 2]
  - Saída Gerada: 2 (Resultado OK)
  - Apenas 1 caso de Teste Satisfez o Critério



Técnicas de Teste Estrutural Critérios Estruturais Clássicos

- Todos-Arcos
  - Exige que os casos de teste exercitem todos os arcos pelo menos uma vez
  - Se todos os Arcos forem Executados, todos os nós terão sido executados pelo menos uma vez
  - Critério Todos-Arcos inclui o Critério Todos-Nós
  - Conclusão: O critério Todos-Arcos é mais forte que Todos-Nós

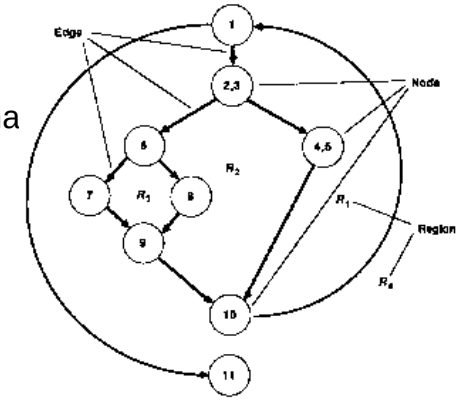


- Exemplo de Todos-Arcos
- Arcos Requeridos para satisfazer o critério:
  - (1,2); (1,3); (2,3); (3,4); (4,5); (5,4); (4,6)
- Casos de Teste Entrada Caminho Completo Executado Saída Esperada
  - #1 [(4,30) , (1,2,3,4,5,4,6) , 2]
    Saída Gerada: 2 (Resultado: OK)
    85% de cobertura, pois não cobre o arco (2,3)
  - #2 [(10,3) , (1,3,4,5,4,6) , 1]
    Saída Gerada: 1 (Resultado: OK)
    Cobre o arco restante, atingindo 100% cobertura

- **Critério Todos-Caminhos-Independentes**
  - Baseado na Análise de Complexidade Ciclomática do Código, de McCabe
  - Garante que todos os comandos (nós do GFC) e todos os ramos (arcos do GFC) serão cobertos
  - Caminho Independente é qualquer caminho do programa que introduza pelo menos um novo conjunto de instruções de processamento ou uma nova condição
  - Caminhos Independentes forma um conjunto básico que pode ser usado para derivar casos de teste mais detalhados

Exemplo de Caminhos Independentes:

- Grafo Derivado do Fluxograma
  1. 1-11
  2. 1-2-3-4-5-10-1-11
  3. 1-2-3-6-8-9-10-1-11
  4. 1-2-3-6-7-9-10-1-11



- Todos incluem, pelo menos, 1 novo arco
- O Caminho 1-2-3-4-5-10-1-2-3-6-8-9-10-1-11 não é independente (é uma simples combinação dos caminhos 2 e 3)

- **Caminhos Independentes:**

a) 1,2,3,4,6

b) 1,3,4,6

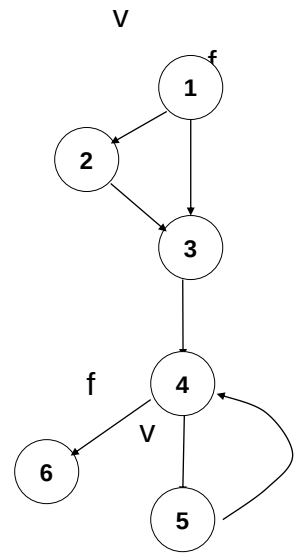
b) 1,2,3,4,5,4,6

- **Conjunto Básico não é único**
- **Outro Conjunto possível:**

a) 1,2,3,4,6

b) 1,3,4,5,4,6

- Aplicação do conjunto Máximo aumenta a confiabilidade do conjunto de casos de teste



Técnicas de Teste Estrutural Critérios Estruturais Clássicos

- Métrica da Complexidade Ciclomática
  - V(G) = Arcos - Nos + 2, ou
  - V(G) = Nós Predicativos + 1, ou
  - V(G) = Numero de Regiões do Grafo

- V(G) representa maior cardinalidade do conjunto Básico
- Métrica de Dificuldade Quantitativa do Teste Estrutural de Unidades (ex. Módulos)
- Estudos de McCabe indicam que V(G) <= 10 é um limite de capacidade de gestão da complexidade



Técnicas de Teste Estrutural Critérios Estruturais Clássicos

- Avaliar os nós de predicado (condicionais)
- Projetar casos de teste que, dado um conjunto de entrada, sensibilize todo o caminho que deseja-se exercitar.
- Não esquecer de calcular a saída esperada
  - Teste estrutural não é simplesmente para cobrir 100% elementos do grafo de um programa
  - Bom caso de teste é aquele que expõe uma nova falha
- Use Caminhos Básicos para detalhar o teste



Técnicas de Teste Estrutural Análise de Eficácia

- Revelam cerca de 5% dos defeitos
  - 35%: Teste de Caminhos
  - 65%: Teste de caminhos combinados com teste de laços (teste de caminhos estendidos à ciclo)
- Há Problema da Abordagem Estrutural Pura
  - Cobre, no melhor caso, algumas sequências de condições do programa (caminhos do GFC)
  - Teste de caminhos não consideram aspectos ligados ao processamento do programa, ou seja como os dados são definidos, usados e modificados conforme fluem no processamento do programa



Técnicas de Teste Estrutural
Estrutural Puro x Exaustivo

Teste Estrutural		**Teste Exaustivo**
Todos Nós ou Arcos Todos Camin-Indep. Crit. estendido a ciclo	**"GAP"** **ABISMO**	Todos Caminhos

Como diminuir o GAP??

- Testar o comportamento dos dados através do fluxo de controle (caminhos) do programa, afim de revelar anomalias
- Utilizar Critério de Teste estrutural baseados em Análise de Fluxo de Dados



- Analisa como os objetos de dados (variáveis, constantes, objetos instanciáveis, etc.) do programa são tratados através dos vários caminhos do GFC
- Critérios de teste estruturais baseados em Fluxo de Dados fornecem medidas de cobertura que servem como guia para:
  - Efetividade do teste
  - Adequação (quando parar)

- Terminologia (Estados dos objeto de dados)
  - **Definição**: ocorre quando um valor é armazenado em uma posição da memória
  - Exemplos

    a = 1; (definição da variável a)

    b = b + 1; (definição da variável b)

    read (**x**); (definição da variável x)

- Terminologia (Estados dos objeto de dados)
  - **Uso**: ocorre quando um objeto de dados é referenciado em um programa. Dois tipos:
    - P-Uso (uso Predicativo): em condições (if, while)
    - C-Uso (uso Computacional) => atribuições, saídas, expressões aritméticas, etc.
  - **Exemplos**:

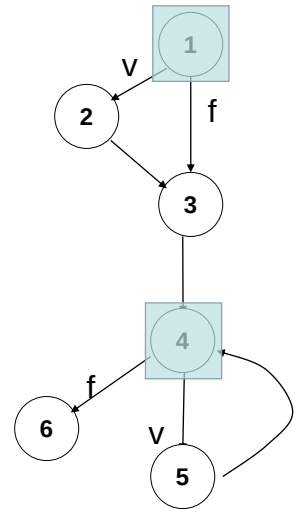
    a = **g**; (c-uso da variável g e definição de a)
    b = **b** + 1; (c-uso e definição de b)
    if (a>b) .... (p-uso das variáveis a e b)
    write (x); (uso da variável x)

- Requerem que sejam testadas as interações que envolvam definições de variáveis e subsequentes referências (usos) a essas variáveis no grafo.
  - **Nó inicial** – onde há definição de uma variável
  - **Nó final** – onde ocorre o uso da mesma variável
- As Interações entre o nó inicial e o nó final são os Elementos Requeridos pelo teste estrutural orientado pelo fluxo de dados, em categorias:
  - **Caminhos** – Sequencia estrita e direta do nó inicial ao nó final de uma variável do programa (nó a nó do Grafo e sem quebra da sequência)
  - **Associações** – Qualquer caminho possível entre um nó inicial e um nó final de uma variável do programa (Grafo)

Técnicas de Teste Estrutural Exemplo de GFC

Associação (1, 4, p/ var <r>) - Nó 1 definição de r; Nó 4 uso de r

```
Nó Comandos
1 int mdc(int a, int b)
1 { int r=0,aux;;
1 if (a < b) // inverte a e b
2 { aux = a;
2   a = b;
2   b = aux; }
3 r = a % b;
4 while (r!=0)
5 {   a = b;
5     b = r;
5     r = a % b; }
6  return b;
6 }
```

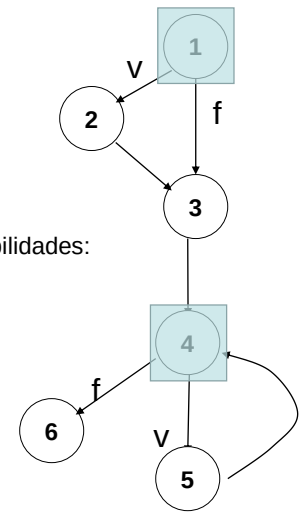


Técnicas de Teste Estrutural Exemplo de GFC

Associação (1, 4, p/ var <r>) - Nó 1 definição de r; Nó 4 uso de r

```
Nó Comandos
1 int mdc(int a, int b)
1 { int r=0,aux;;
1 if (a < b) // inverte a e b
2 { aux = a;
2   a = b;
2   b = aux; }
3 r = a % b;
4 while (r!=0)
5 {   a = b;
5     b = r;
5     r = a % b; }
6  return b;
6 }
```


Duas possibilidades:
1 – 2 – 3 - 4
1 – 3 - 4



Técnicas de Teste Estrutural Exemplo de GFC

Alguns Caminhos Def-Uso <r> 4-5; 3-4-5; 5-4-5;

```
Nó Comandos
1 int mdc(int a, int b)
1 { int r=0,aux;;
1 if (a < b) // inverte a e b
2 { aux = a;
2   a = b;
2   b = aux; }
3 r = a % b;
4 while (r!=0)
5 {   a = b;
5     b = r;
5     r = a % b; }
6  return b;
6 }
```

